

Case Study: The E-Commerce Black Friday

Table of Contents

Requirements	3
Expected Volumes	3
Functional Requirements	3
High-level Architecture Considerations.....	3
1. Systems and Subsystems Required	4
2. Types of Information Exchanged	6
4.Failure Scenarios	9
5. Mitigation Actions.....	9
6. Difference Between Average Load and Peak Load.....	10
7.Deployment Diagram	11
8. Cost-Effective Architecture Decisions	12

Requirements

An e-commerce company is launching a new platform to support high-traffic events like Black Friday. The new system must be resilient and scalable to provide seamless transactions.

Expected Volumes

1. Expected Daily Active Users: 1 million
2. Peak Concurrent users: 250,000
3. Transactions per second (TPS): 10,000
4. Supported platforms: web, mobile (IOS & Android)

Functional Requirements

1. Secure user authentication (SSO, MFA)
2. Product catalogue, shopping cart and checkout
3. Multiple payment gateway integrations with failover.
4. Order tracking, notifications and user reviews.
5. Admin dashboard with analytics and role-based access.

High-level Architecture Considerations

1. What are the various systems and subsystems required to deliver this functionality?
2. What kind of information is exchanged (frequency, speed, volume etc.)?
3. What are potential failure points?
4. What kind of mitigation actions are necessary to ensure failure preventions (highlight is as reactive, proactive or preventive)?
5. Do you see a difference in the way the system needs to behave during the average load and peak load?
6. What would you architect for knowing that there can be cost implications?

1. Systems and Subsystems Required

To support a scalable, resilient e-commerce system for Black Friday-level traffic, as an architecture we should include the following **modular systems and subsystems**:

Core Systems:

- **Frontend Platforms**
 - Web (React)
 - Mobile Apps (iOS, Android - built with React Native/Flutter)
- **Authentication System**
 - SSO (OAuth 2.0, OpenID Connect)
 - MFA (TOTP, SMS, Email OTP)
- **Product Management**
 - Product Catalog Service (with search and filtering)
 - Inventory Management System
- **Shopping Cart Service**
 - Session-based or user-profile-based carts
 - Cart persistence and sync
- **Checkout System**
 - Pricing engine, discounts, shipping calculations
- **Payment Gateway Integrations**
 - Multiple providers (Stripe, PayPal, Razor-Pay, etc.)
 - Payment Orchestrator with automatic failover
- **Order Management System**
 - Order lifecycle tracking
 - Fulfillment coordination
- **Notification System**
 - Email, SMS, Push (via services like Twilio, Firebase)
- **User Feedback System**
 - Review, rating, and moderation module
- **Admin Dashboard**
 - Analytics, order stats, user engagement

- Role-based access control (RBAC)
- **Logging & Monitoring**
 - Centralized logging (ELK, Datadog)
 - Distributed tracing (Open-Telemetry, Jaeger)
 - Alerts & anomaly detection
- **Database Systems**
 - Relational DBs for transactions (PostgreSQL, MySQL)
 - NoSQL for product catalog (MongoDB/DynamoDB/Cassandra)
 - Caching layer (Redis, Memcached)
- **Infrastructure & DevOps**
 - Containerization (Docker, Kubernetes)
 - Auto-scaling (AWS/GCP/Azure)
 - CI/CD pipeline
 - API Gateway, CDN (Cloudflare, Akamai)

2. Types of Information Exchanged

Breakdown of **data types, exchange frequency, speed, and volumes**:

1. User Authentication & Session Data

- **Data Type:** JWTs, OAuth tokens, MFA codes, user profiles (PII).
- **Frequency: High:** 1M+ daily logins (peaking at 250K concurrent).
- **Speed:** <100ms latency for SSO/MFA flows.
- **Volume:** ~50KB per session (token + encrypted user data).

2. Product Catalog

- **Data Type:** Product metadata (names, prices, descriptions), inventory counts, images.
- **Frequency: Extreme:** 10K+ reads/sec (peaks), writes ~100/sec (admin updates).
- **Speed:** Cache hits (Redis): <5ms, DB reads (MongoDB): <20ms.
- **Volume:** ~1TB total (images dominate), cache holds ~10GB hot items.

3. Shopping Cart

- **Data Type:** Item IDs, quantities, user session IDs.
- **Frequency: Spiky:** 50K+ updates/sec (add/remove items during peaks).
- **Speed:** Redis-backed cart: <10ms write latency.
- **Volume:** ~100MB/sec transient data (ephemeral storage).

4. Checkout & Payments

- **Data Type:** Order details, payment tokens (PCI-compliant), fraud scores.
- **Frequency: Critical:** 10K TPS sustained during peaks.
- **Speed:** Payment processing: 200-500ms (gateway latency).
- **Volume:** ~1GB/min order data (Cassandra writes).

5. Order Fulfillment

- **Data Type:** Shipping addresses, tracking IDs, status updates.
- **Frequency: Bursty:** 5K+ updates/sec (status changes).
- **Speed:** DB writes: <50ms (Cassandra optimized).
- **Volume:** ~500MB/hour (logs + DB entries).

6. Notifications

- **Data Type:** Emails (order confirmations), SMS (OTPs), push notifications.
- **Frequency: Async:** 50K+ notifications/hour (peaks).
- **Speed:** Queue processing: <1 sec (Kafka/SQS).

- **Volume:** ~10MB/min (text-heavy).

7. Analytics & Monitoring

- **Data Type:** Clickstream data, logs, metrics (CPU, latency).
- **Frequency: Continuous:** 100K+ events/sec (Prometheus).
- **Speed:** Near real-time (1-5 sec delay for dashboards).
- **Volume:** ~10GB/day (compressed logs).

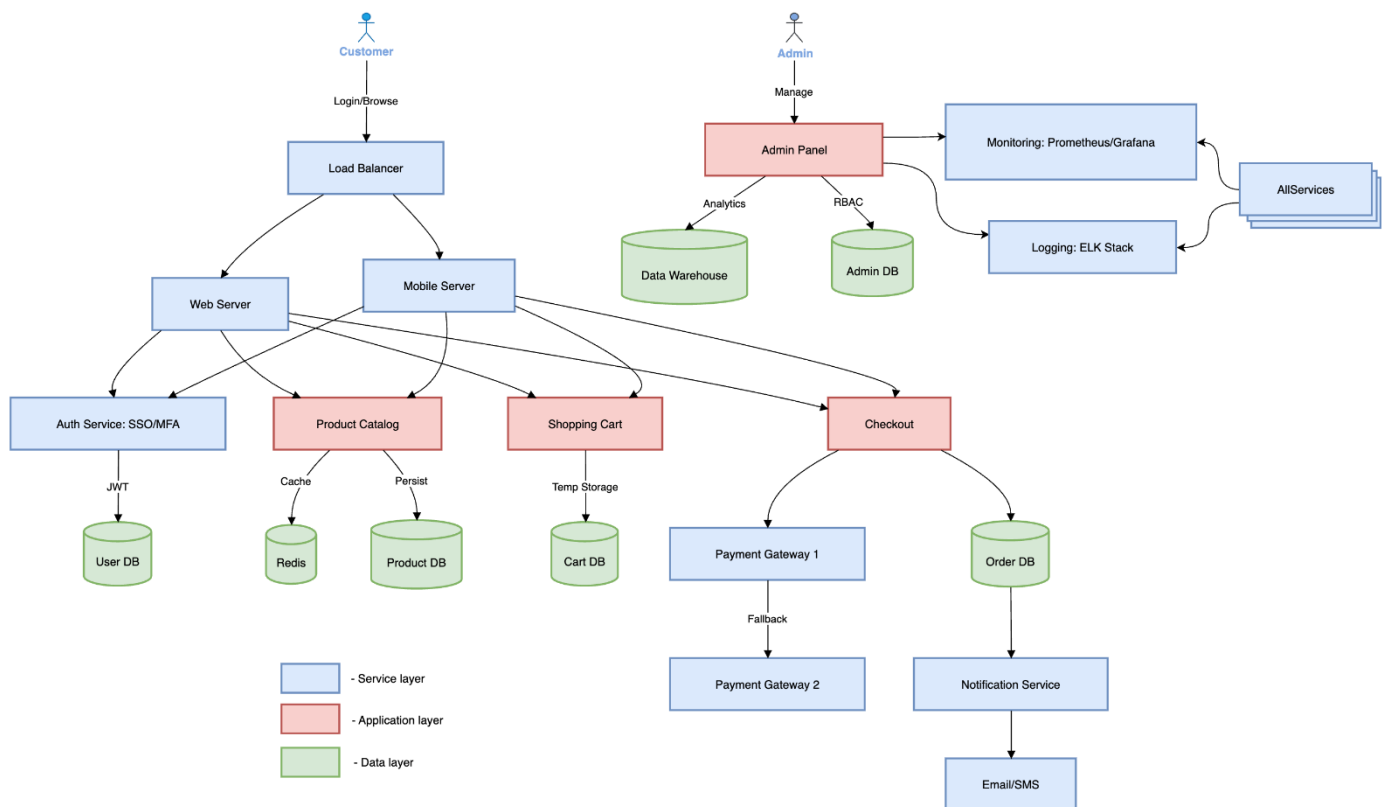
Data Flow Summary

Component	Data Type	Frequency	Speed Requirement	Volume (Peak)
User Auth	Tokens, PII	1M+/day	<100ms	50GB/day
Product Catalog	JSON, Images	10K reads/sec	<20ms (DB)	1TB storage
Shopping Cart	Session data	50K updates/sec	<10ms	100MB/sec transient
Payments	PCI tokens	10K TPS	<500ms	1GB/min
Notifications	Emails, SMS	50K/hour	<1 sec async	10MB/min
Analytics	Logs, Metrics	100K events/sec	Near real-time	10GB/day

Key Challenges & Solutions

Key Area	Challenge	Solution
Peak Traffic Handling	10K TPS payment processing	Cassandra (horizontal scaling) + idempotent APIs
Low-Latency Reads	Product catalog access at scale	Redis cache (95% hit rate)
Data Compliance	CI/PII protection	Tokenization (Stripe), encryption (AWS KMS)
Monitoring Overhead	100K events/sec logging	ELK stack with log sampling

3. Components and Data Flow Diagram



Customers interact via Web/Mobile → Load Balancer distributes traffic.
Auth Service handles SSO/MFA securely.

1. Core Features:
 - Product Catalog: Cached (Redis) + DB-persisted.
 - Shopping Cart: Session storage with auto-expiry.
 - Checkout: Connects to Payment Processor with failover.
2. Resilience:
 - Database Cluster: Master-slave replication.
 - Payment Gateways: Fallback mechanism.
 - Microservices: Decoupled for scalability.
3. Admin & Monitoring:
 - Dashboard: Role-based access to analytics.
 - Monitoring: Prometheus + ELK for real-time logs.
 - Redundancy: All critical components (DB, Payment) have backups.
4. Scalability:
 - Horizontal Scaling: Stateless services auto-scale.
 - Peak Handling: 10K TPS via distributed systems.
 - Traffic Spikes: Load balancer + caching handle 250K concurrent users.
 - Low Traffic Mode: System scales down 100x via cloud auto-scaling.

4.Failure Scenarios

Component	Failure Scenarios
Authentication	Latency or downtime in SSO or MFA provider
Cart Service	State loss or inconsistency between sessions
Payment Gateway	Gateway downtime or transaction failure
Product Catalog	High latency on DB queries or search index corruption
Order Service	Deadlocks, duplicate orders, stuck order states
Notification Service	Failed delivery, spam filters
Infrastructure/Scaling	Insufficient autoscaling or hitting rate limits
CDN or Cache	Cache-miss, stale data, propagation delays
Database	Connection saturation, slow queries
Admin Dashboard	Incorrect role access or slow analytics queries

5. Mitigation Actions

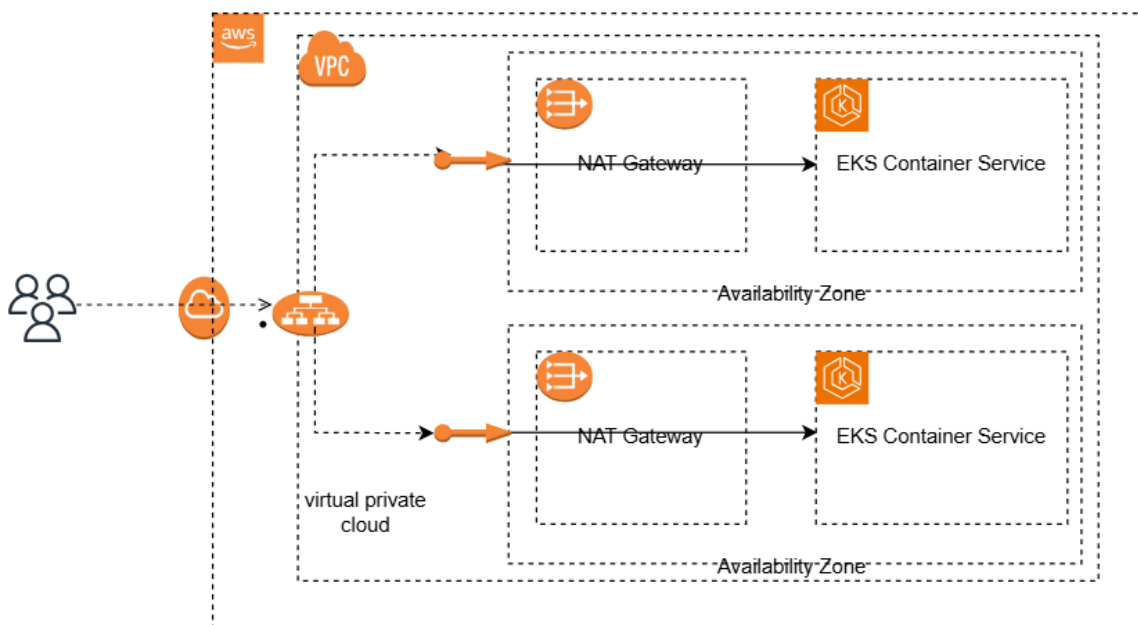
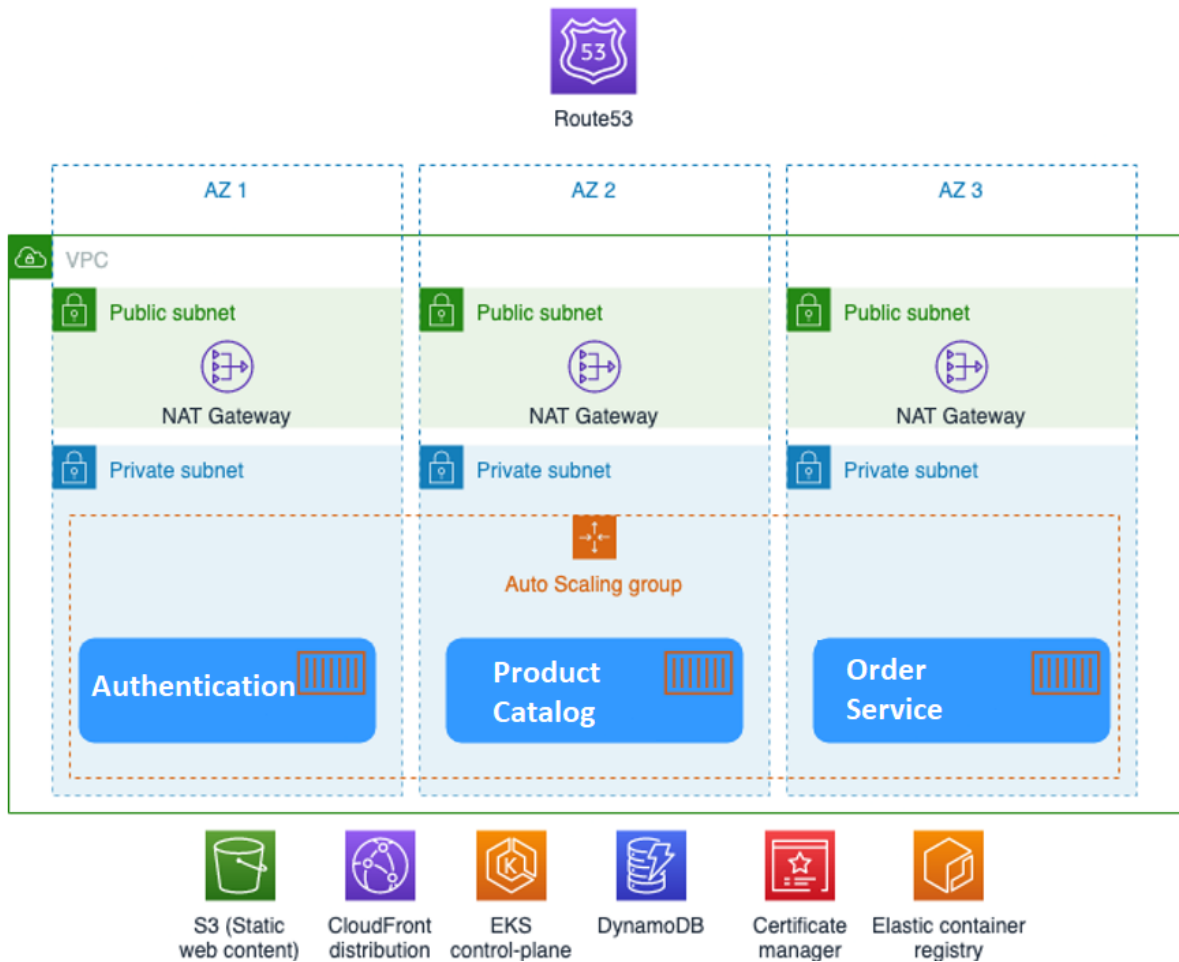
Component	Action Type	Mitigation
Auth & MFA	Proactive	Use redundant auth providers, fallback MFA methods
Payment Gateway	Reactive	Retry logic, auto-failover between gateways
Caching Layer	Preventive	Warm up cache, lazy loading strategies
Database	Preventive	Query optimization, read replicas, sharding
Cart & Checkout	Proactive	Session stickiness, service retries
Infrastructure	Preventive	Auto-scaling, chaos testing
Notification	Reactive	Retry queues, dead-letter handling
Monitoring	Proactive	Real-time alerts, dashboards for ops team
System Updates	Preventive	Canary deployments, blue-green deployments

6. Difference Between Average Load and Peak Load

Yes, the system must handle **drastic behavioral shifts** during peak load:

Aspect	Average Load	Peak Load (Black Friday)
Traffic volume	Moderate & predictable	Sudden spikes, high concurrency
Resource allocation	Minimum instances auto-scaled	Pre-warmed resources, over-provisioned buffers
Caching	Normal cache usage	Cache hit ratio must be near 100%
Queues & Async ops	Lower throughput	High throughput, resilient queuing
Monitoring	Lower priority alerts	Aggressive real-time monitoring
Rate-limiting	Basic controls	Strict rate limits to prevent overload
Feature toggles	Full features	Might disable less-critical features (e.g., reviews)

7. Deployment Diagram



8. Cost-Effective Architecture Decisions

Given cost constraints, focus on **balancing performance and cost**:

Smart Choices:

- Use **serverless** components (e.g., AWS Lambda, Firebase Functions) for bursty workloads (e.g., notifications).
- **Cache everything**: Use CDN + Redis for product listings, static assets, and cart items.
- Use **message queues** (Kafka, SQS) for async and retrievable tasks like notifications and order processing.
- Employ **multi-tier storage**: Premium SSD for transactional DBs, cheaper blob storage for logs/media.
- **Auto-scaling** Kubernetes or ECS clusters—scale-to-zero where possible.
- Leverage **spot instances** or **reserved instances** for predictable workloads.
- Integrate **feature flags** to gracefully degrade non-essential services during high load.

9. Security Considerations

Here are the critical security considerations for your high-traffic e-commerce system, designed to handle Black Friday-scale loads while protecting data and ensuring compliance:

1. Infrastructure Security

DDoS Protection:

- Use AWS Shield/Cloudflare to mitigate volumetric attacks during peak traffic.
- Deploy rate limiting (e.g., 10,000 TPS per IP).

Network Segmentation:

- Isolate public-facing servers (web/mobile) from internal services (DBs, payment) using VPCs/NACLs.

WAF (Web Application Firewall):

- Block SQLi, XSS, and bot traffic (e.g., AWS WAF with OWASP rules).

2. Data Security

Encryption:

- In Transit: TLS 1.3 for all communications (HTTPS, API calls).
- At Rest: AES-256 for databases (e.g., PostgreSQL, Cassandra).

PCI DSS Compliance:

- Tokenize credit card data; never store raw PANs. Use PCI-compliant gateways (Stripe, PayPal).

Data Masking:

- Anonymize PII in logs/analytics (e.g., replace emails with hashes).

3. Authentication & Access Control

Multi-Factor Authentication (MFA):

- Enforce MFA for admin dashboards and sensitive operations (e.g., refunds).
- Role-Based Access Control (RBAC):
- Limit admin access (e.g., "read-only" for support staff).

Session Management:

- Short-lived JWT tokens (15–30 mins) with Redis-backed revocation.

4. Payment Security

Payment Gateway Failover:

- Use certified providers (Stripe + Braintree) with automatic fallback.

Fraud Detection:

- Integrate tools like Signifyd or Kount for real-time fraud scoring.

Idempotency Keys:

- Prevent duplicate charges during retries (critical for 10K TPS).

5. API Security

OAuth 2.0/OpenID Connect:

- Secure mobile/third-party API access with scoped permissions.

API Rate Limiting:

- Throttle non-critical endpoints (e.g., product reviews) during peaks.

6. Monitoring & Incident Response

Real-Time Alerts:

- Monitor for anomalies (e.g., 500% spike in failed logins).

Audit Logs:

- Track all admin actions (who deleted a product?).
- Automated Responses:
- Auto-block IPs after 10 failed auth attempts.

7. Compliance & Legal

GDPR/CCPA:

- User data deletion workflows. Cookie consent banners.

SOC 2 Type II:

- Required if handling enterprise customers.

8. Scalability-Specific Risks

Cache Poisoning:

- Sanitize keys in Redis (e.g., user input as cache keys).

Database NoSQL Injection:

- Validate inputs for MongoDB/Cassandra queries.

Auto-Scaling Misconfigs:

- Ensure new instances boot with hardened AMLs/containers.

Implementation Checklist:

1. Enable DDoS protection (AWS Shield/Cloudflare).
2. Encrypt all databases (AWS KMS/HashiCorp Vault).
3. Tokenize payments via Stripe/Braintree.
4. Deploy WAF with OWASP Core Rule Set.
5. Test failover for payment gateways.
6. Schedule quarterly penetration tests.

Example Attack Mitigations

Threat	Solution
Card skimming	PCI DSS + iFrame embeds (Stripe.js)
Credential stuffing	MFA + CAPTCHA for >5 failed logins
Inventory hoarding bots	Rate limit "Add to Cart" API calls